01 - MiniMind 项目介绍话术

> 面试中被问到「介绍一下你做过的项目」时，根据面试节奏选择合适的版本。

一、30 秒版本（电梯演讲）

> 适用于：群面自我介绍、面试开场快速破冰

话术：

我独立完成了一个名为 MiniMind 的大语言模型全链路训练项目。这是一个 64M 参数的 Decoder-Only Transformer 模型，架构对齐 Qwen3，使用纯 PyTorch 从零实现。我在单张 3090 显卡上，仅花费约 3 元和 2 小时，就完成了从 Tokenizer 训练、预训练、SFT 微调、LoRA、DPO/PPO 对齐到 MoE 扩展的完整训练链路。整个项目让我深入理解了 LLM 从数据准备到模型部署的每一个环节，包括 GQA 注意力、RoPE 位置编码、RMSNorm、SwiGLU 激活等现代 LLM 核心技术。

二、1 分钟版本（常规面试）

> 适用于：技术面试中 "介绍一个印象最深的项目"

话术：

我做了一个叫 MiniMind 的大语言模型全链路训练项目。这个项目的目标是用最低的成本，从零实现一个完整的 LLM 训练流程。

在模型架构上，我实现了一个 64M 参数的 Decoder-Only Transformer，对齐 Qwen3 的设计。具体来说，使用了 GQA 分组查询注意力（8 个 Q 头、4 个 KV 头）、RoPE 旋转位置编码、Pre-Norm + RMSNorm 归一化方案，以及 SwiGLU 激活函数。模型的隐藏维度是 768，共 8 层，词表大小 6400，并采用了 Embedding 和 LM Head 的权重共享策略。

在训练流程上，我实现了完整的六阶段训练链路：首先训练了一个 6400 词表的 BPE Tokenizer；然后进行 Next Token Prediction 预训练；接着进行 SFT 有监督微调，使用了 loss_mask 只计算 assistant 部分的损失；之后实现了 LoRA 参数高效微调；再通过 DPO 和 PPO/GRPO 进行人类偏好对齐；最后还扩展了 MoE 混合专家模型、Tool Calling 能力，以及从 Qwen 系列进行知识蒸馏。

整个项目用纯 PyTorch 原生实现，在单张 3090 上仅花费约 3 元和 2 小时即可完成训练，让我对 LLM 的每个技术细节都有了深入理解。

三、3 分钟版本（深入介绍）

> 适用于：终面/项目深挖环节

话术：

我来详细介绍一下 MiniMind 这个项目。

项目背景与动机： 当前开源大模型虽然效果很好，但动辄几十亿甚至上千亿参数，普通开发者很难从底层理解它的训练全流程。我做这个项目的初衷，就是用最小的规模——64M 参数——来复现一个「麻雀虽小五脏俱全」的 LLM，覆盖从 Tokenizer 到 RLHF 对齐的完整链路，同时保证每一行代码都是纯 PyTorch 实现，没有使用 HuggingFace Trainer 等高层封装。

模型架构设计： 架构上采用 Decoder-Only Transformer，对齐 Qwen3 的设计方案。注意力机制使用 GQA（Grouped Query Attention），配置 8 个 Query 头和 4 个 KV 头，相比标准 MHA 减少了约 50% 的 KV 缓存开销。位置编码使用 RoPE（Rotary Position Embedding），rope_theta 设为 1e6，并支持 YaRN 长度外推方案。归一化采用 Pre-Norm + RMSNorm，相比 Post-Norm + LayerNorm 既提升了训练稳定性又减少了计算量。激活函数使用 SwiGLU，FFN 的中间维度通过 SwiGLU 的特殊结构进行调整。模型参数配置为 d_model=768、n_layers=8、vocab_size=6400，并且 embed_tokens 和 lm_head 共享权重以减少参数量。

训练流程实现： 训练分为多个阶段。第一步是 Tokenizer 训练，我基于 BPE 算法训练了一个 6400 大小的词表。第二步是预训练，使用 Next Token Prediction 目标，配合 warmup + cosine annealing 学习率调度、混合精度训练和梯度累积策略。第三步是 SFT 微调，关键技术点是构建 loss_mask，只对 assistant 回复部分计算交叉熵损失，并支持多轮对话的 chat_template 格式。第四步实现了 LoRA 参数高效微调，将低秩矩阵注入到注意力层的 QKV 投影中。第五步是偏好对齐，分别实现了 DPO 和 PPO/GRPO 两种方案——DPO 直接利用偏好数据对优化策略，不需要单独训练奖励模型；PPO 则通过奖励模型和 clip 机制来约束策略更新。

进阶扩展： 在基础训练之上，我还实现了几个扩展功能：MoE 混合专家模型，使用 Top-K 路由和负载均衡 loss 来防止专家坍缩；Tool Calling 能力，让模型能够调用外部工具；以及从 Qwen 系列大模型到 MiniMind 的知识蒸馏，包含黑盒蒸馏（软标签 KL 散度）方案。

项目成果与收获： 最终，整个项目在单张 3090 显卡上仅需约 3 元成本和 2 小时即可完成全流程训练。通过这个项目，我不仅掌握了 LLM 训练的每个技术细节，还对大模型的架构设计、训练稳定性、数据工程等问题有了深入的实践认知。

四、简历描述模板

简洁版（1-2 行）

> 从零实现 64M 参数 Decoder-Only Transformer（对齐 Qwen3），使用纯 PyTorch 完成 Tokenizer→Pretrain→SFT→LoRA→DPO→PPO/GRPO→MoE→蒸馏全链路训练，单卡 3090 训练成本仅 ¥3/2h。

标准版（3-4 行）

> **MiniMind —— 64M 参数大语言模型全链路训练** - 基于纯 PyTorch 从零实现 Decoder-Only Transformer，对齐 Qwen3 架构（GQA、RoPE、RMSNorm、SwiGLU） - 完成 Tokenizer(BPE)→预训练(NTP)→SFT→LoRA→DPO→PPO/GRPO 全阶段训练，并扩展 MoE、Tool Calling、知识蒸馏 - 单卡 3090 完成全流程训练，成本 ¥3/2h；深入理解 KV-Cache、混合精度训练、梯度累积等工程优化

详细版（5-6 行）

> **MiniMind —— 64M 参数大语言模型全链路训练（纯 PyTorch 实现）** - 设计并实现 Decoder-Only Transformer 架构，对齐 Qwen3：GQA（q=8,kv=4）、RoPE（theta=1e6,支持 YaRN 外推）、Pre-Norm+RMSNorm、SwiGLU 激活，d_model=768，8 层，vocab=6400，embed/lm_head 权重共享 - 完成 BPE Tokenizer 训练 → Next Token Prediction 预训练 → SFT 微调（loss_mask + chat_template）→ LoRA 参数高效微调 → DPO/PPO/GRPO 偏好对齐完整训练链路 - 扩展实现 MoE 混合专家（Top-K 路由 + 负载均衡 loss）、Tool Calling、Qwen→MiniMind 知识蒸馏（黑盒 KL

> 散度） - 工程优化：混合精度训练（AMP）、梯度累积、梯度裁剪、warmup+cosine 学习率调度、KV-Cache 推理加速、DDP 多卡训练 - 单卡 3090 完成全部训练，成本 ¥3/2h；纯 PyTorch 原生实现，无 HuggingFace Trainer 依赖

五、面试技巧提示

场景	建议
面试官说「简单介绍一下」	用 30 秒版本，讲完后主动问「需要展开讲某个部分吗？」
面试官说「详细介绍一下」	用 1 分钟版本，注意观察面试官表情，有兴趣则展开
面试官说「这个项目具体怎么做的」	用 3 分钟版本，注意按逻辑分段，每段之间可以稍作停顿
面试官追问某个技术点	参考后续的面试题文档，进入深度技术讨论
简历空间有限	用简洁版，但确保关键词（Transformer、GQA、RoPE、SFT、DPO、LoRA）都出现

核心亮点要反复强调： 1. **全链路**：从 Tokenizer 到 RLHF，不是只做了其中一步 2. **从零实现**：纯 PyTorch，不是调用 HuggingFace API 3. **低成本**：¥3 + 2 小时 + 单卡 3090，体现工程能力 4. **对齐主流**：架构对齐 Qwen3，不是自己瞎设计的 5. **技术深度**：能展开讲每个组件的原理和实现细节